Master's Thesis

Type-Driven Type-Correct Code Suggestions for Incorrect Functional Programs

With implementation for a toy language.

Sijmen van Bommel
s1038820

July 13, 2025

First supervisor/assessor:
dr. P.M. Achten

Second assessor:
dr. M. Lubbers

Radboud University

Abstract

When a programmer makes a mistake, the state-of-the-art error messages can tell you what and where something has caused a conflict the compiler found. However, they do not tell you how to fix it. We provide a system that generates type-correct suggestions. These are generated deterministically without use of generative AI.

Contents

Chapter 1

Introduction

Programmers make mistakes. Luckily, we have error messages that can help programmers for certain kinds of mistakes. For most functional languages, a compiler error can occur in one of the following phases of a compiler: Lexical, Parsing, Typing, Codegen [Hage et al., 2006]. We focus on the parsing and typing phases. An error in the parsing phase can tell us if the structure of the code is correct; it will detect mistakes in, e.g., indentation, parenthesis, and the usage of the right symbols. An error in the typing phase can tell us if our program makes sense based on the type system. For example, it will find that [1]

```
1 not :: Bool -> Bool
2
3 fun = not 4
```

does not make sense since the `not` function expects a boolean and not an integer. Modern compilers spend a lot of effort in telling you exactly what has gone wrong, and often even try to explain why. There are, however, still cases where this is not always what is desired. Take, for example, a fault for the typing phase:

```
1 plus :: Int -> Int -> Int
2
3 fun = plus plus 4 5 6
```

here the problem is that the programmer missed the parenthesis around `plus 4 5`. The typing phase will see this mistake because, as it is written now, we tell it to supply the function `plus` as the first argument `plus`, which

[1] We use Haskell syntax for the examples. As a convention, type signatures of functions are given and assumed to be correct without requiring implementation.

expects an integer, not a function. This results in a type error:

```
Couldn't match expected type 'Int'
            with actual type 'Int -> Int -> Int'
```

While this is exactly what the problem is, it requires the user to figure out that the missing parentheses were the issue. In addition to this error explaining what went wrong, we propose additionally giving a suggestion like so:

```
did you mean:
fun = plus (plus 4 5) 6
--diff:--
fun = plus (plus 4 5) 6
--Which has type: Int
```

As a second example, take a fault in the parsing phase:

```
plus :: Int -> Int -> Int

trice :: (a -> a) -> a -> a
trice f x = f (f (f x))

fun = trice (plus 4 5
```

here we forgot a closing parenthesis. This will result in a parse error:

```
parse error (possibly incorrect indentation or mismatched brackets)
fun = trice (plus 4 5
                     ∧
```

Note here that the ∧ is in the wrong place; the closing parenthesis should go between the 4 and the 5 since trice takes a function as an argument, which can only be `plus 4` (partially applied). This is expected because the parser does not know what `trice` is. Only in the typing phase will we know this. Parsers will only detect that a closing parenthesis is missing and suggest you place it at the end. Following this suggestion would result in a new error now in the typing phase. From the perspective of the programmer, this is really strange. The compiler first asks you to put a parenthesis somewhere, after which it will complain it is in the wrong spot. Instead, we want a suggestion like:

```
did you mean:
fun = trice (plus 4) 5
--diff:--
fun = trice (plus 4) 5
--Which has type: Int
```

In this thesis, we will show how to generate these suggestions.

1.1 Problem Statement

When generating these suggestions, what is the problem we are actually solving? We give the program some text that does not compile. The goal is to only give suggestions that compile and that have a minimal number of transformations on the original text.

The underlying assumption is that programmers tend to write mostly correct code. So if the programmer makes one mistake, the fix for this mistake will probably be the same as the smallest number of transformations on the malformed code to make it compile (and therefore type check).

What a transformation is is intentionally left a bit vague. A simple metric, such as the number of characters or tokens added/removed, makes for a good start. But we should also consider other transformations, such as the swapping of arguments to a function. If a programmer gives arguments in the wrong order, you might need to change a lot of individual tokens to swap the arguments. But one can also see this swapping as one transformation.

In short, we want an algorithm that can take malformed input and generate compiling code with minimal transformations. Such an algorithm that can guarantee the smallest number of transformations would be the holy grail of making suggestions.

Unfortunately, realistic requirements get in the way. Going over all possible permutations (to guarantee minimal transformations) is simply too slow [Lee et al., 2018][Campos et al., 2021]. In this thesis, we try to get close to this holy grail but also make decisions to keep a reasonable runtime, as discussed in Chapter 9. We do this by letting the suggestions be guided by the type system. For this guidance, we assume that type annotations by the user can be trusted. They can be used to guide how suggestions are generated.

1.2 Motivation

[Becker et al., 2019] states that there is a lot of improvement to be made for compiler error messages. As [Hage et al., 2006] and [Tirronen et al., 2015] state, this is even more the case for beginner programmers. We also see that

type and parsing errors are the ones most often encountered by beginner programmers. (Type and parsing errors are the kinds of errors we can make suggestions for.) This is also reflected by the author of this thesis's personal experience being a teaching assistant for the introductory 2nd-year bachelor functional programming course at the Radboud University. We believe that both finding the smallest number of transformations to make code compile and our implementation that tries to do that for certain kinds of errors, will help in bettering some of these common type and parser errors.

1.3 Objectives

We have some soft and hard objectives. Soft objectives are the things our algorithm strives to do. The hard objectives are concrete tests that our algorithm should pass.

1.3.1 Soft Objectives

One mistake. We assume that programmers write mostly correct code. Ideally we would detect all mistakes a programmer made, but we are aiming for being able to fix all code with one mistake.

What we classify as a mistake will change this metric. We want to be able to fix any one of the following mistakes:

1. Forgetting an opening/closing parenthesis.
2. Forgetting to put an expression into parentheses.
3. Giving arguments to a function in the wrong order.
4. Using the wrong symbol for `->` in a lambda abstraction.
5. Messing up indentation in let expressions.

Runtime. If we have a look at program synthesis, [Lee et al., 2018] claims that some generative algorithms are too slow. We often target simple mistakes with our suggestions, so it is no use if we give a suggestion after the user has spotted and solved the problem themselves. The search space for generating all possible changes in order to find the minimum number of changes that will make the program compile, is simply too large. Therefore, we aim for a reasonably quick runtime, see Chapter 9.

1.3.2 Hard Objectives

Suggestions must compile. We require that any suggestion generated with our algorithm will compile. Meaning that if we accept the suggestion,

we would not have to change that code to get a compiling program. If this is not possible, we should not generate a suggestion.

This assures that the user does not get into a loop where accepting one suggestion leads to a new suggestion, which leads to a new suggestion, etc. The user only compiles once and either gets a suggestion which will work or does not get a suggestion.

The exception is that it may be the case that accepting a suggestion exposes a new error in some other function, but never in the function we accepted the suggestion for. We do not generate a suggestion when a function is used for which the type is unknown. Accepting a suggestion can make a type known, so if we accept a suggestion for function A, we might get a new suggestion for function B for which no suggestion could be generated a s it used function A.

Leaving correct code untouched. There are multiple ways of writing the same thing. Suggestions should not suggest changing from one notation to an identical one. We take a working function and introduce a fault at the end of it. The suggestion builder should not change the beginning of that function, even if this new suggestion would also work. We propose a test:

$$\text{Suggestion}(\texttt{str} + +\texttt{"("}) = \texttt{str} \qquad \text{if compiles}(\texttt{str})$$

for some notion of equality. So if we take a string that compiles and we add a `(` to it, we introduce an error in the parsing phase. The given suggestion for this code should be the original string, again for some notion of equality.

Our notion of equality allows for changes in indentation (e.g., 4 spaces instead of the original 5) and the removal of empty lines.

1.4 Contributions

We provide the following contributions:

- We define algorithms for generating always type-checking suggestions based on the types.
- A small core language to demonstrate as a demonstrator and proof of concept for our algorithm.
- A proof of concept Implementation of these algorithms, the output of which can be seen in Chapter 2 and the code in Appendix A.
- An algorithm for displaying the suggestions to the user.
- Instructions on how to integrate these suggestions into an existing compiler in Chapter 6.
- Analysis on the algorithm performance in Chapter 9

1.5 Structure of the Thesis

We will start off by giving examples of what our algorithm can currently do in Chapter 2. Then after some preliminaries, Chapter 3, and related work, Chapter 4, we go over the language definition of the language we can make suggestions for and the current restrictions on said language in Chapter 5. In Chapter 6 we go over the requirements for a compiler that wants to incorporate these suggestions. How the suggestions are generated can be found in Chapter 7. And in Chapter 8 We cover how we display our suggestion to the user. In Chapter 9 we go over the performance of our system. We finish off with the conclusion, Chapter 10, reflection and future work, Chapter 11. The code for generating the suggestions can be found in the appendix.

Chapter 2

Examples

In this chapter, we show representative examples of malformed inputs and the suggestions we can currently generate (for the code, see appendix A). Here we use code from our core language that we defined in Chapter 5. We use the convention that type definitions may be given without implementation, where we assume such an implementation exists and is correct. The blue colouring is auto-generated to aid readability. The green and red colouring is a direct output of the system (see Chapter 8).

We start with a small showcase, combining mistakes to show some extreme examples of what the suggestion builder can handle. After that, we will go over different kinds of fixes/language structures and give some examples for those.

2.1 Showcase

Example 2.1.1.

We start with a common case of missing parentheses:

```
plus :: Int -> Int -> Int

fun = plus plus 4 5 6
```

Output:

```
did you mean:
fun = plus (plus 4 5) 6
--diff:--
fun = plus (plus 4 5) 6
--Which has type: Int
```

Example 2.1.2.

An extreme version with missing parentheses and arguments being applied in the wrong order:

```
invertNum :: Bool -> Int -> Int

plus :: Int -> Int -> Int

fun = invertNum invertNum plus plus 4 5 plus plus plus 6 7 8 9  True False
```

Output:

```
did you mean:
fun = invertNum False (invertNum True (plus (plus 4 5) (plus (plus (plus 6 7) 8)
    9)))
--diff:--
fun = invertNum False (invertNum True (plus (plus 4 5) (plus (plus (plus 6 7) 8) 9
    ))) True False
--Which has type: Int
```

Example 2.1.3.

A poorly formatted lambda expression:

```
iterate :: (Int -> Int) -> Int -> (Int -> Int)

plus :: Int -> Int -> Int

fun = iterate \x plus 5 x 8 2
```

Output:

```
did you mean:
fun = iterate (\x -> plus 5 x) 8 2
--diff:--
fun = iterate (\x -> plus 5 x) 8 2
--Which has type: Int
```

Example 2.1.4.

Let expression chaos:

```
plus :: Int -> Int -> Int

fun x y =
    let
        fun a b = plus a b
            var = let
                plus x y = x in plus x True
                myConst = 5

        plus fun myConst var y)
```

Output:

```
did you mean:
fun x y =
    let
        fun a b = plus a b
        var =
            let
                plus x y = x
            in plus x True
        myConst = 5
    in
        plus (fun myConst var) y
--diff:--
fun x y =
    let
        fun a b = plus a b
        var = newline
        let
                plus x y = x newline
                in plus x True
        myConst = 5
    in
        plus (fun myConst var) y)
--Which has type: (Int -> (Int -> Int))
```

2.2 Parenthesis

Example 2.2.1.

We remove unnecessary parentheses:

```
plus :: Int -> Int -> Int

fun = plus 2 4)
```

Output:

```
did you mean:
fun = plus 2 4
--diff:--
fun = plus 2 4)
--Which has type: Int
```

Example 2.2.2.

We respect superfluous parentheses:

```
plus :: Int -> Int -> Int

fun = (plus 2) 4)
```

Output:

```
did you mean:
fun = (plus 2) 4
--diff:--
fun = (plus 2) 4)
--Which has type: Int
```

Here the parentheses around `plus 2` could be removed without changing the meaning, they are superfluous. Respecting these especially helps in showing the difference, as it now only shows a difference for the parenthesis that caused the problem.

Example 2.2.3.

We suggest parentheses such that the types match. This is a case where suggestions that originate from parsers often wrongly suggest a closing parenthesis at the very end:

```
trice f x = f (f (f x))

plus :: Int -> Int -> Int

fun = trice (plus 2 4
```

Output:

```
did you mean:
fun = trice (plus 2) 4
--diff:--
fun = trice (plus 2) 4
--Which has type: Int
```

Example 2.2.4.

We restore parentheses in a deeply nested manner:

```
plus :: Int -> Int -> Int

fun = plus plus plus 4 5 plus plus plus 2 3 4 5 6
```

Output:

```
did you mean:
fun = plus (plus (plus 4 5) (plus (plus (plus 2 3) 4) 5)) 6
--diff:--
fun = plus (plus (plus 4 5) (plus (plus (plus 2 3) 4) 5)) 6
--Which has type: Int
```

We are not limited by the number of mistakes, only the sort of mistake.

Example 2.2.5.

We can make suggestions for partial applications:

```
plus :: Int -> Int -> Int

fun = plus plus 4 4
```

Output:

```
did you mean:
fun = plus (plus 4 4)
--diff:--
fun = plus (plus 4 4)
--Which has type: (Int -> Int)
```

2.3 Argument order

Example 2.3.1.

We can suggest changing the order of arguments:

```
invertNum :: Bool -> Int -> Int

fun = invertNum 6 True
```

Output:

```
did you mean:
fun = invertNum True 6
--diff:--
fun = invertNum True 6 True
--Which has type: Int
```

Example 2.3.2.

We can do this in a nested manner:

```
invertNum :: Bool -> Int -> Int

fun = invertNum invertNum 6 True False
```

Output:

```
did you mean:
fun = invertNum False (invertNum True 6)
--diff:--
fun = invertNum False (invertNum True 6) True False
--Which has type: Int
```

Note how The suggestion puts `False` first. One might imagine a greedy approach that just takes the first boolean it finds. Then one would expect the `True` to come first. The suggestion builder may treat `invertNum` like it has a different type with the argument order changed. We only swap arguments at the very end. We do this so our suggestion remains consistent with misremembering the type of `invertNum`.

Example 2.3.3.

We reorder any number of arguments:

```
myChar1 :: Char
myChar2 :: Char

multipleArgumentFunction :: Bool -> Char -> Int -> Int -> Bool -> Char -> Bool

fun = multipleArgumentFunction myChar1 myChar2 True 4 5 False
```

Output:

```
did you mean:
fun = multipleArgumentFunction True myChar1 4 5 False myChar2
--diff:--
fun = multipleArgumentFunction True myChar1 myChar2 True 4 5 False myChar2
--Which has type: Bool
```

We are not limited by the number of arguments.

2.4 Lambdas

Example 2.4.1.

We fix parentheses for lambdas:

```
plus :: Int -> Int -> Int

fun = \x -> plus x x 5
```

Output:

```
did you mean:
fun = (\x -> plus x x) 5
--diff:--
fun = (\x -> plus x x) 5
--Which has type: Int
```

Example 2.4.2.

We handle lambda expressions with multiple arguments:

```
plus :: Int -> Int -> Int

fun = \x y -> plus x y 5
```

Output:

```
did you mean:
fun = (\x y -> plus x y) 5
--diff:--
fun = (\x y -> plus x y) 5
--Which has type: (Int -> Int)
```

Example 2.4.3.

We respect alternative notation for lambdas with multiple arguments:

```
plus :: Int -> Int -> Int

fun = \x  -> (\y -> plus x y 5
```

Output:

```
did you mean:
fun = (\x -> (\y -> plus x y) 5)
--diff:--
fun = (\x -> (\y -> plus x y) 5)
--Which has type: (Int -> Int)
```

Example 2.4.4.

We support nested lambdas:

```
trice f x = f (f (f x))

plus :: Int -> Int -> Int

fun = trice \x -> trice \y -> plus y y x 5
```

Output:

```
did you mean:
fun = trice (\x -> trice (\y -> plus y y) x) 5
--diff:--
fun = trice (\x -> trice (\y -> plus y y) x) 5
--Which has type: Int
```

Example 2.4.5.

We give suggestions when using the wrong symbol for `->`:

```
trice f x = f (f (f x))

plus :: Int -> Int -> Int

fun = trice \x = plus 5 x 8)
```

Output:

```
did you mean:
fun = trice (\x -> plus 5 x) 8
--diff:--
fun = trice (\x -> = plus 5 x) 8)
--Which has type: Int
```

Example 2.4.6.

We can figure out where the `->` should go if the number of arguments is known from the required type:

```
iterate :: (Int -> Int) -> Int -> (Int -> Int)

plus :: Int -> Int -> Int

fun = iterate \x plus 5 x 8 2
```

Output:

```
did you mean:
fun = iterate (\x -> plus 5 x) 8 2
--diff:--
fun = iterate (\x -> plus 5 x) 8 2
--Which has type: Int
```

2.5 Let expressions

Example 2.5.1.

We make suggestions for let statements. Both for local definitions and the let body:

```
plus :: Int -> Int -> Int

invertNum :: Bool -> Int -> Int

fun =
    let
        res = plus plus 4 5 6
    in  invertNum res True
```

Output:

```
did you mean:
fun =
    let
        res = plus (plus 4 5) 6
    in invertNum True res
--diff:--
fun =
    let
        res = plus (plus 4 5) 6
    in invertNum True res True
--Which has type: Int
```

This shows that let expressions do not hinder in generating suggestions within it.

Example 2.5.2.

We have suggestions where local definitions depend on each other:

```
plus :: Int -> Int -> Int

fun x =
    let
        fun3 = plus x fun2
        fun2 = plus plus 5 6 7
    in
        fun3
```

Output:

```
did you mean:
fun x =
    let
        fun3 = plus x fun2
        fun2 = plus (plus 5 6) 7
    in
        fun3
--diff:--
fun x =
    let
        fun3 = plus x fun2
        fun2 = plus (plus 5 6) 7
    in
        fun3
--Which has type: (Int -> Int)
```

Example 2.5.3.

We handle weird indentation:

```
fun =
    let
        fun3 = 4
            fun2 = 5
              fun4 = True
    in
        fun3
```

Output:

```
did you mean:
fun =
    let
        fun3 = 4
        fun2 = 5
        fun4 = True
    in
        fun3
--diff:--
fun =
    let
        fun3 = 4
        fun2 = 5
        fun4 = True
    in
        fun3
--Which has type: Int
```

Example 2.5.4.

We handle nested lets:

```
fun =
    let
        var3 =
            let
                fun3 = plus
            in
                fun4 5 (fun3 5 6)
                fun4 a b = plus a b
    in
        var3
```

Output:

```
did you mean:
fun =
    let
        var3 =
            let
                fun3 = plus
            in
                fun4 5 (fun3 5 6)
        fun4 a b = plus a b
    in
        var3
--diff:--
fun =
    let
        var3 =
            let
                fun3 = plus
            in
                fun4 5 (fun3 5 6)
        fun4 a b = plus a b
    in
        var3
--Which has type: Int
```

Example 2.5.5.

We figure out where the `in` goes if the last local definition cannot take more arguments:

```
fun x =
    let
        fun2 = plus 4 x
    fun2
```

Output:

```
did you mean:
fun x =
    let
        fun2 = plus 4 x
    in fun2
--diff:--
fun x =
    let
        fun2 = plus 4 x
    in fun2
--Which has type: (Int -> Int)
```

2.6 Miscellaneous

Example 2.6.1.

We use type annotations to guide the suggestions:

```
invertNum :: Bool -> Int -> Int

fun :: Int -> Bool -> Int

fun x y = invertNum x y)
```

Output:

```
did you mean:
fun x y = invertNum y x
--diff:--
fun x y = invertNum y x y)
--Which has type: (Int -> (Bool -> Int))
```

Here we see how the type given by the user can influence how the suggestion is generated. If we ignored it here, we would not have known that swapping arguments was required to make the definition match the type.

We have seen different examples of problems we can generate suggestions for. We would like to note that these problems can be combined. There are also cases where we cannot generate a suggestion. We will only ever show a suggestion if we know it will compile.

Chapter 3

Preliminaries

In this Chapter, we will cover subjects one needs to know to read this thesis at the level required for this thesis.

3.1 Haskell

Haskell is a language originally made to unify the differing functional programming languages between universities [Hudak et al., 2007].

We do not require intimate knowledge of Haskell for the majority of this thesis, as we specify our language in Chapter 5. The language we use is heavily inspired by Haskell 98 [Jones, 2003] and we reference it a few times. So it is recommended to have a grasp of the basics. Haskell is also the language used for the implementation of our suggestions (see appendix A).

Chapter 7 is the exception. We give explanations without requiring intimate Haskell knowledge. However, we also go over some implementation details in Haskell. That requires knowledge about more advanced subjects like Maybes, state monads, and more.

We also mention algorithmic data types (ADTs) a few times. These are used to make your own types; these are out of scope for this thesis.

3.2 Hindley-Milner Type System

Haskell uses an extension of the Hindley-Milner Type System. [Heeren, 2005] has an excellent preliminaries chapter on the exact details of the type system. Here we give a short recap, focusing on the parts important to this thesis.

A type describes how a function/constant looks and how it can be used. For example `4`, `5` and `42` are all `Int`s [1] and `True` and `False` are both `Bool`s.

[1] Numbers in Haskell are a bit more complicated, as they are all overloaded. For our purposes, it is sufficient to think of numbers as intergers.

For describing functions, we introduce an `->`. A function of the type `Int -> Bool` gets an `Int` as input and produces a `Bool`. If we want to describe functions that take multiple arguments, like `plus` we do this like so: `plus :: Int -> (Int -> Int)` here the `::` means "has type". The parentheses indicate that we do not have a function that takes two arguments but a function that takes one and produces a new function. This function can, in turn, consume one more argument. In the world of types, there is no such thing as a function that takes two arguments. Although this notion of functions returning functions is in a way equivalent to having functions that can take multiple arguments.

The `->` is right associative, so `Int -> (Int -> Int)` may be written as `plus :: Int -> Int -> Int`. Do note that `plus :: (Int -> Int) -> Int` is meaningfully different, as this describes a function that takes an entire function as input.

We can also have type variables, denoted with lowercase letters, usually `a b c d e` etc. The usage for type variables is quite simple: a type variable can be any type (even a function) but if you plug something in at one place, you must plug it in for all other instances of that type variable.

3.3 Compilers

A compiler has one primary job: converting some text (a string) into code that can either be run directly on hardware or compiled again by a different compiler. The secondary function of a compiler is informing the user if and why this generation of code might fail. It can detect problems before any user code is run. This secondary task of informing the user is usually implemented as error messages.

A compiler usually works in a few phases: tokeniser, parser, typing, and code generation. We cover these in order below.

3.3.1 Tokeniser

A Tokeniser, also known as a Lexer, has the job of converting the input text into tokens.

A token is a sequence of characters that has meaning. The primary idea of a token is to group relevant information together for the parser.

As an example:

```
fun = "hello world"
```

Here we have nineteen individual characters. A tokeniser can convert it into just three tokens:

```
[Name "fun", EqualsSign, StringLiteral "hello world"]
```

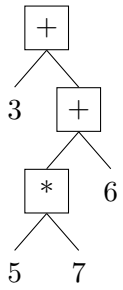
It groups the relevant information together. This makes it more manageable for the parser.

One other function a tokeniser has, is that it can fail if it encounters a symbol that it does not know. This can be useful feedback, as an unknown symbol is probably a mistake.

Some tokenisers can additionally give some more information about whitespace and indentation. For example, in this thesis, we use a tokeniser that removes whitespace information and replaces it with indent and dedent tokens.

3.3.2 Parser

A parser's job is to convert a list of tokens or token stream into an abstract syntax tree (AST). An AST is a way to see expressions as a tree. For example, the AST of the expression `3 + 5 * 7 + 6` as an AST would look like:



Such a tree is much easier to manipulate for compiler-related algorithms than a line of text. The AST for most programming languages is much more complex than the example above, including logical structures such as functions and conditionals, etc.

A parser makes this conversion by consuming one token at a time and matching it to patterns encoded in it. The exact details are not needed for this thesis.

We do need to know that this parsing might fail. For example, the expression `3 + + 5` can not be parsed. Whenever this happens, we get a

parse error. Parse errors usually tell where the parser got stuck and what it was expecting at that place.

3.3.3 Typing

In Section 3.2 we have seen what a type is. In the context of a compiler, we usually talk about type environments, type checking and type inferencing.

The typing phase makes sure the program adheres to the typing rules. If it does not, this indicates a logical error and compiling should fail.

Type Environment

A type environment is a mapping from (function) names to types. It tells you how each name can be used. It can happen that in the same programme, the same name has different meanings at different places. This is because the type environment can change throughout a program. We can even have two different definitions with the same name, as long as one is more specific. Here the more specific one shadows the binding. meaning that the more specific one would be used over the more generic one.

Type Inferencing

Type inferencing is the process of seeing which type some expression has. It will need the current type environment to do so. It can take an expression like `4 < 6` and tell that it is an `Bool`.

For this thesis, it is not required to know how this works. For those interested, [Heeren, 2005] has an excellent preliminaries Chapter covering type inference in detail.

This type inference/checking might fail. Not all expressions are typeable. When this happens, we get a type error explaining where we expected a certain type and what we got instead.

Type Checking

Type checking is seeing if some expression/AST has a certain type. To do this, it needs the current type environment so it can look up the types of any named functions it may encounter. For example, we can check that `4 + 6` is in fact, an integer. But if we do the same for `4 < 6` this, it will fail. To explain why it fails, we need to know what the type of `4 < 6` actually is. This brings us back to type inferencing.

Type checking is often implemented by just inferring the type of the expression and then checking if the types are similar. What we mean by similar is that there is a most general unifier (mgu). A most general unifier is a substitution for the type variables such that after applying it to both

types, they are the same. For example, an expression of type `a -> a` can be unified with the type `Int -> Int`. Because we can replace the `a` by `Int`.

3.3.4 Code generator

After we have made our tokens, parsed them into an AST and checked that the types work with the inferencer, we can end by converting the AST into code either for hardware directly or some target language.

This thesis focuses on the error messages, so we put generating the code out of scope.

3.4 Error Messages

Error messages are used to inform the user when something has gone wrong. We focus on compile-time errors. These are the errors you get while compiling.

The goal of an compile-time error message is to communicate to the user that something about compiling has gone wrong and why it has gone wrong. Such an error can originate from one of the different phases of a compiler. Generating a good error message is difficult [Becker et al., 2019]. They often require knowledge from the user about the compiler phases to be understood. Compile-time error messages tend to explain what has gone wrong in a compiler phase. In this thesis, we do not focus on generating such errors. We focus on adding suggestions to them. A suggestion is some code snippet telling the user how to fix the problem; it does not focus on the why.

3.5 Terminology

We will elaborate on some terminology used:

- Malformed tokens: A sequence of tokens that will not parse and/or type-check. In this thesis, we usually mean tokens resulting from code with one or two mistakes, be it incorrect parenthesis/indentation or the swapping of arguments. We rarely mean completely random tokens.
- User: We take the perspective of the compiler. So a user is a programmer who gives input code to the compiler.

Chapter 4

Related Work

The related work can be put in roughly three categories: error messages, code synthesis and code competition. We discuss these in detail below. Each of these areas all touch our topic, yet our topic lies in between them.

Research on error messages matches our research in that error messages are often integrated deeply in the compiler. They have access to information from the compiler phases and use this to display and localise the errors. Where it differs from our topic is that we can make guesses in our suggestions; we may fail. Another difference is that we generate our suggestions outside the normal compiler phases, taking information from multiple phases of the compiler. Most fundamentally, error messages have a different goal. They tell users what has gone wrong. We tell users how they might fix it. The suggestions are complementary to the error messages.

Research on code synthesis matches our research in that we try to generate code that will fit in a certain program. Where it differs is that code synthesis often depends on the user when and where code needs generating. This is usually in the form of test cases or holes. Code synthesis systems are usually not designed to work with partially incorrect code. We generate our suggestions exclusively on incorrect code. We decide where to generate suggestions based on the compiler alone. We require no input from the user to generate our suggestions.

Research on code completion matches our research in that it is designed to work on incomplete/incorrect code. They can incorporate type information to generate suggestions. Where it differs is in where the suggestions are generated; this usually only happens where (or around where) the user is typing. It also does not have all the information available we have by directly integrating in the compiler.

4.1 Error Messages

There has been plenty of work on error messages. [Rhemrev, 2023] goes over the attributes for a good error message, with a focus on Java errors.

[Becker et al., 2019] collects a lot of research into error messages and highlights common shortcomings. These both lay emphasis on the difficulties for novice programmers. [Tirronen et al., 2015] focuses specifically on beginners and the kinds of mistakes they make. It shows that parsing and type errors are most prevalent. For more related work on error messages specifically, see the related work Chapter from [Lee et al., 2018].

4.2 Code Synthesis

Our suggestions are a bit like code synthesis; where synthesis differs is in its reliance on test cases and "holes" like in [Lee et al., 2018]. They propose a system for fixing logical errors for functional programming assignments. [Campos et al., 2021] proposes a system that based on some test cases, can give real-time suggestions to JavaScript code.

[Raychev et al., 2014] covers a slightly different approach to code synthesis. They introduce a system where a "hole" can be added in your code, and their tool will try to fill it in with working code.

Recently, [Popovici, 2023] explores using ChatGPT in a functional programming course. They find that instead of synthesising code, its strengths lie in the reviewing process, claiming "77% of feedback having correct reasoning".

4.3 Code Completion

Somewhat related, here is some research on code completion, "the auto complete suggestions you get while typing." [Bruch et al., 2009]. These autocomplete suggestions are built up out of the keywords and functions available. [Bruch et al., 2009] builds a relevance metric using types and information they got by analysing existing code bases. It uses this relevance metric to reorder the usual list of all possible suggestions to set the most relevant code first.

[Franks et al., 2015] Uses an AI language model to make a similar ordering for autocomplete suggestions.

Chapter 5

Language Definition and Restrictions

In this chapter, we define the language we can make suggestions for. Suggestions are generated one malformed function at a time.

If the suggestions are implemented in a bigger language. Then these language restrictions only apply to malformed functions. These restrictions do not apply to correct functions in said bigger language (see section 6.5).

We will be targeting Haskell (see Chapter 3.1) syntax.

5.1 Grammar

We first give a context-free grammar. In Section 5.2 till Section 5.3 we describe the grammar in more detail. We omit indentation from our grammar:

$$
\begin{aligned}
\text{Program} &::= \text{Function} \mid \text{TypeDef} \mid \text{Program Program} \\
\text{Function} &::= \text{Name Arguments} = \text{Expr} \\
\\
\text{Expr} &::= \text{Name} \mid \text{Expr Expr} \mid (\text{Expr}) \mid (\lambda\, \text{Arguments} \to \text{Expr}) \mid \text{let } \{\text{LocalDef}\} \text{ in Expr} \\
\text{LocalDef} &::= \text{Name Arguments} = \text{Expr} \\
\text{Arguments} &::= \{\text{Name}\} \\
\\
\text{TypeDef} &::= \text{Name} :: \text{Type} \\
\text{Type} &::= \text{BaseType} \mid \text{Type} \to \text{Type} \mid (\text{Type}) \mid \text{TypeVar} \\
\text{TypeVar} &::= \texttt{Name} \\
\text{BaseType} &::= \texttt{Int} \mid \texttt{Bool} \\
\\
\text{Name} &::= \texttt{AscSmall\{AscSmall\}} \\
\text{AscSmall} &::= \texttt{a} \mid \texttt{b} \mid \cdots \mid \texttt{z}
\end{aligned}
$$

This grammar is sufficient as a core to show how we generate suggestions.

5.2 Constants, Functions, Applications, and Types

The most basic language consists of function applications and constants.

Constants We implement booleans and integers for our constants. All constants are essentially handled the same. Adding more (characters, strings, floats, etc.) is highly recommended but is not relevant to our research goals and is therefore left out of scope.

Function Definitions We define functions like in Haskell (see Chapter 3.1) which consist of some function name followed by 0 or more arguments, then an equals sign and finally, some expression. For example:

```
1 functionName arg1 arg2 arg3 = 42
```

Function applications To write interesting expressions for our functions, constants are not enough; we want to be able to use other functions and supply them with arguments. For example

```
1 fun = not True
```

We use parentheses to group applications:

```
fun x = (plus x) ((plus 5) 6)
```

Function application is left associative, so we may omit some parentheses:

```
fun x = plus x (plus 5 6)
```

Types Finally we specify types by first giving a name, then a `::` keyword and finally a type (see section 3.3.3) like so:

```
fun :: Int -> Int
```

Since our focus is on generating suggestions, we allow having a function being defined by only a type without an implementation. For building a suggestion, only the types of the functions that are available are required. This is useful for testing the suggestions, but is obviously problematic for code generation. Code generation is out of scope for this thesis, but one should not allow this in compilers that do.

5.3 Lambda Expressions

Lambda expressions allow you to define a function inside an expression. This is a relatively small addition to the syntax: simply a `\` followed by 1 or more arguments, then an `->` keyword and finally, some expression, just as with functions. (We do view function definitions and lambda expressions as separate.)

```
fun x = (\y -> plus x y)
```

They are handled similarly to functions but introduce two interesting aspects. The first is argument scoping; we allow for one of the arguments to shadow (use the same name as and overwrite) existing functions. The second is that lambdas with more than one argument can be written in different ways, e.g.

```
fun  = (\x -> (\y -> plus x y))
```

is equivalent to

```
fun  = (\x y -> plus x y)
```

5.4 Let Expressions

Let expressions allow you to define multiple functions in a restricted scope. It is a `let` followed by an list of functions that all start on the same indentation. We call these the local definitions. What follows is a `In` keyword followed by the expression where said list of functions is available. For example:

```
fun  =
    let
        myNum = 5
        mySecondNum = 7
        myResult = plus myNum mySecondNum
    in
        plus 4 myResult
```

Most notably, this adds indentation-specific syntax that can be nested and functions that can (order independently) depend on another.

5.5 Restrictions

There are some notable restrictions to our current implementation that warrant justification.

infix operators We do not currently support infix operators, meaning expressions like `4 + 5` are not supported. This decision was made to reduce scope. We do think they could be supported to some extent, although not straightforwardly; see section 7.2.1.

recursive functions Recursive functions are currently unsupported, unless the type for the recursive function is given by a type annotation. The problem is that in order to process a recursive call, we need to know the type of the function. But we can only figure out the type by generating the suggestion, which in turn needs the type of the recursive call. One can probably give a more generic type if a loop is detected and then later check

if the specific type fits. Since we thought this would not cause major issues, this was placed out of scope.

5.6 Notable Exclusions

There are some aspects missing from our language that one might expect from a Haskell toy language. Below, we reflect on the potential and tradeoffs for some of these concepts.

If Statements We do not foresee any problems adding support for if statements. They can almost be defined as functions of the type `if :: bool -> a -> a -> a` already, and the addition of the words `then` and `else` in the syntax should not pose any major problems, even when nested.

ADTs, Case statements and pattern matching Algebraic data types (ADTs), and most notably case statements and pattern matching, are out of scope. For defining ADTs we do not see how type driven suggestions will give better hints than parsers do currently, as they have few restrictions outside syntax. Case statements and pattern matching could hugely benefit from suggestions, as there are ample opportunities for misplaced parentheses and indentation.

If one assumes correct structure for case statements, it would be quite straightforward to give the extra type information case statements/pattern matching gives you to the suggestion builder. Then suggestions could be made for expressions contained in the case statement. The problems arise when you need to give suggestions on indentation and pattern matching. This would require some further research to figure out.

At first glance, the problem with indentation becomes even greater when considering case statements can be nested. And unlike let statements, case statements do not have a `In` keyword to nicely signal the end of their scope. Here we choose to adhere to existing syntax conventions, instead of adding a closing keyword to scope case statements. Fortunately, our current implementation of let statements does not require an `In` keyword to be present in order for our suggestion builder to work. This was an intentional decision made in preparation for adding case statements.

List Notation Lists may be constructed using the operators in their algorithmic data type (ADT). However, functional languages often use syntax with square brackets and comma separation. e.g.

```
1 [True, False, False]
```

there are a lot of simple mistakes one could imagine suggestions for, missing commas, missing a square bracket, forgetting to make a value a singleton list, etc.

We think doing these kinds of suggestions is compatible with our current approach. But more research is needed on this. For this thesis, this is out of scope.

Chapter 6

Compiler Requirements

In this chapter, we go over what we require from a compiler in order to generate suggestions with our method. See Chapter 7 for how the suggestions are generated.

Note that when we talk about ”the suggestion builder”, we refer to the current implementation; see Appendix A.

6.1 Installing the Suggestion Builder

In Chapter 7 we go into how we generate the suggestions. For now, we treat the so-called ”Suggestion Builder” as a black box.

The purpose of the suggestion builder is simple; it takes a token stream and converts it into an AST. Thus far, it has the same purpose as the parser. Where it differs is that the suggestion builder works with malformed token streams (up to a limit) and always generates an AST that type checks, or does not generate a suggestion at all. The suggestion builder expects the token stream of one function at a time. In order to satisfy the type-checking constraint, we need to give the suggestion builder a type environment containing the types of functions defined outside the given token stream.

So, the suggestion builder takes a malformed token stream and a type environment and generates a type-checking AST (or fails). Finally, we can use this AST to display the differences between what the user typed and the AST we suggest; see Chapter 8.

6.2 Tokeniser and Indentation

To accommodate the suggestion builder, the tokeniser must do a few special things.

In the first place, it should keep the source code location for each token. We recommend this regardless, as it helps in localising errors. But the

suggestion builder needs the location to determine likely indentation on indentation mismatches. As an example, take:

```
some text
 i have one space too many
         I should be on a new indent.
```

The tokeniser will probably see this as:

```
[Text, newLine, Indent, Text, newLine, Indent, Text]
```

Note how there is no difference between the indents. However, one of the indents was one space, and one was several. We want the suggestion builder to see this difference. So we require the location information.

The suggestion builder expects Indent and Dedent tokens. We used the Alex Haskell library [ale,] for tokenising, and it supports this. One tricky case we found is when there is a Dedent but the Dedent does not go back far enough. For example:

```
some text
          one indent
    half a dedent?
```

Here the problem is that on line 2 we Indent but on line 3 we Dedent but not fully. The only logical way to tokenise this is if there are two Indents on line 2. The main problem is that we only realise this later. We solved this issue by adding a special temporary token called InternalAddIndentToPrevious. Then as a postprocessing step, we double up the indent before this token and then remove it, so the example above would yield:

```
[Text, newLine, Indent, Text, newLine, InternalAddIndentToPrevious,Dedent,Text,
   newLine, Dedent]
```

which would then turn into

```
[Text, newLine, Indent, Indent, Text, newLine,Dedent,Text,  newLine, Dedent]
```

This allows the suggestion builder to try to fix the indent instead of failing on the parser.

It is also important for the tokeniser to parse the comments. For the suggestion builder this is not needed, but for displaying the difference be-

tween what was typed and the suggestion, it is important for the comments to be there. We would not want a suggestion to remove all comments.

6.3 Parser

In parsing, there is one key problem. If the parser fails, we do not have an AST to do the type inferencing on. And therefore we cannot give the type environment to the suggestion builder.

To solve this issue, we make sure the parser can only fail on a small section. To do this, we parse in two steps. In the first step, we split the tokens into groups, where each group is split on a new line without indent. This is a fairly simple metric, but it allows us to split off type annotations and function definitions. Here we are assuming malformed functions never miss enough indents to end up as a newline without indents.

In the second step, the parser will go over all these sections and parse them into an AST. This allows for some functions to be malformed while the rest still parses. Then we can collect all the errors and use the separate ASTs to go on to the type inferencing.

6.4 Type Inferencing

Other than allowing for type checking on the separately parsed sections, we take type annotations as truth for sections with functions that do not parse or type check. We only ignore the type annotation for the function we are inferring.

Putting this together, we get figure 6.1.

6.5 Bigger Languages

The suggestion builder currently only supports the language as defined in Chapter 5. However, we are dealing with suggestions; we do not have to give a suggestion. So our algorithm could be used for bigger languages; however, generating a suggestion would fail if constructs are used that are not supported by the suggestion builder. This does not mean that you cannot use these ”unsupported” language constructs, since they can still be used in functions that do not have errors. The suggestion builder only needs the types of correct definitions. So if a language can support a feature up to type inference, as long as it is not used in a malformed function, suggestions could still be generated for that function.

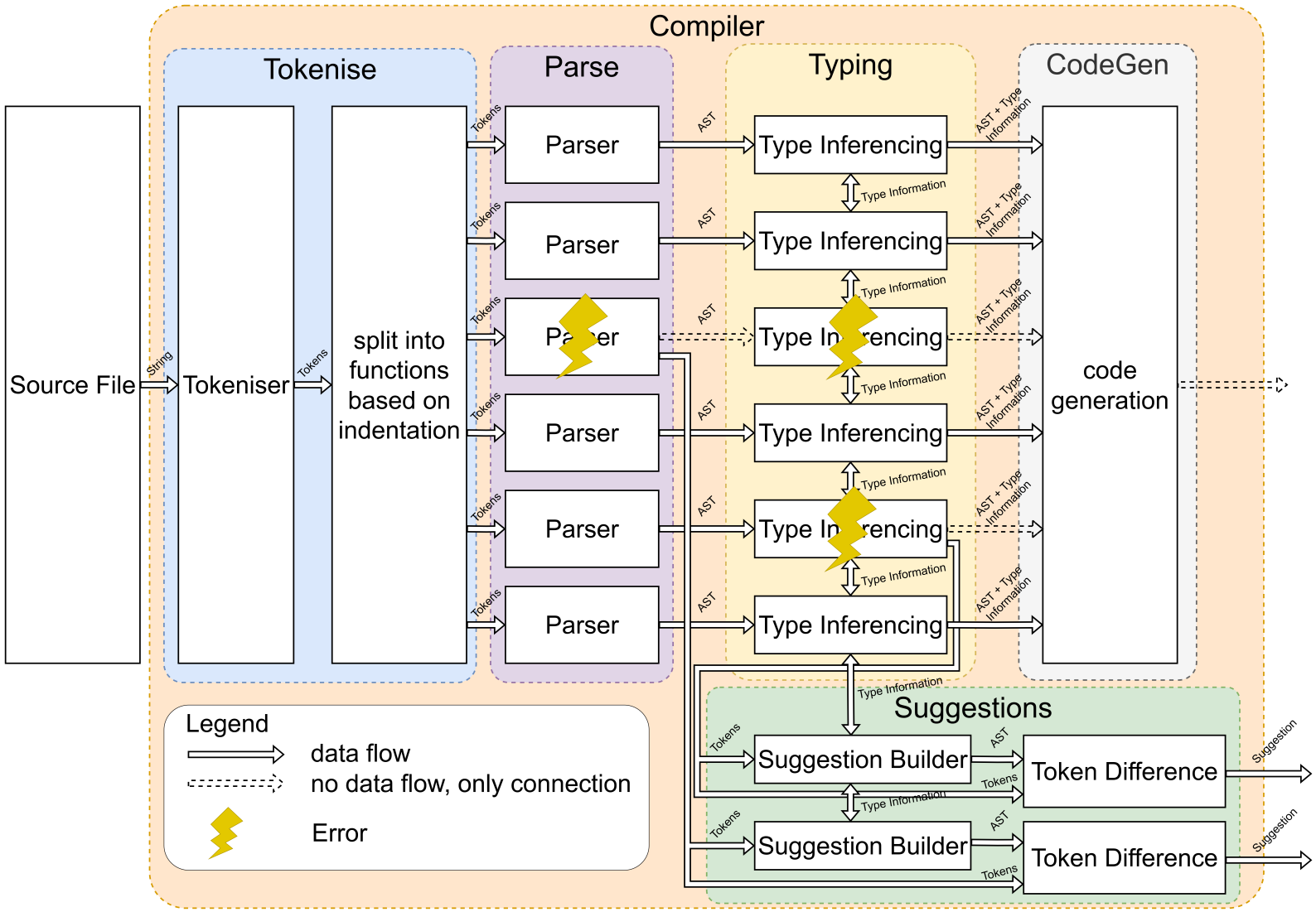


Figure 6.1: Schematic overview of how the suggestions can be integrated into a compiler.

Chapter 7

Suggestion Builder

In this Chapter, we cover how the suggestion builder works. The suggestion builder takes a malformed token stream and the current type environment and generates an AST that type checks. (Also see Chapter 6.1.)

We used an incremental approach to expand the types of suggestions the suggestion builder can make. We will explain the suggestion builder in the same order but incorporate changes required for later parts from the start.

7.1 Reconstructing Parentheses

We start with a language with only constants, functions, applications, and types; see Chapter 5.2.

For some malformed function, we start by assuming only the parentheses are wrongly placed. We start here, as it is a common mistake [Hage et al., 2006] and there are few other things that could go wrong in such a simple language.

We treat the input tokens as if they did not have any parentheses. We want to construct the parenthesis.

7.1.1 Suggestion for Expressions

For now, we focus on the expression part of a function. We cover how to make a suggestion for a whole function in section 7.1.2.

We are able to construct the parentheses based on the types. To demonstrate the idea, we go over an example:

```
1 plus plus 4 5 6
```

The algorithm works by going over the tokens from left to right. So the first thing we see is `plus`. We know that the type of plus is:

```
plus :: Int -> (Int -> Int)
```

So we expect to see an integer next. Instead, we see another `plus`, which is a function and not an `Int`. However, from the type we can see that, given enough arguments, this function can result in an `Int`. So we proceed assuming this second plus should belong together with some arguments in order to produce the desired `Int` and this `plus` should therefore be in parentheses with these arguments. To see how many arguments we need to put in parentheses, we simply continue looking at tokens. We are now plugging in arguments to the second `plus`. So we are looking for an `Int` next. We find a `4` which matches. So we know `plus 4` belong together. The remaning type (after plugging in 4) is `Int -> Int`. This does not yet match the `Int` requested by the first `plus` so we continue. We are once again looking for an `Int`, find a `5` which matches. Now we know that `plus 4 5` belong together. That does have the type `Int` requested by the first `plus`. Thus, we can put it in parentheses and continue with the original plus. `plus (plus 4 5)` has the type `Int -> Int`. So we could take one more `Int`, and there is one, namely `6`. Finally we end up with:

```
plus (plus 4 5) 6
```

This has placed the parenthesis exactly where we desired them.

We implemented this behaviour using a recursive function called `generateExpressionSuggestion`, which has the type:

```
generateExpressionSuggestion ::
    Type -> Maybe Type -> [Candidate] -> SuggestionBuilder [Candidate]
```

To start, we will ignore the `Maybe Type -> [Candidate] ->` and just plug in `Nothing` and an empty list. Doing this we are left with a function that takes a so-called goal and produces a `SuggestionBuilder` for a list of candidates. This `SuggestionBuilder` is a State monad on which we elaborate in section 7.1.3. For now, just think about it like a machine that takes some tokens and a type environment and produces a `[Candidate]`.

The goal is what the suggestion builder tries to make. We often know what type we want and can use this to guide the suggestions. For example, if we have the goal of one `Int` but we encounter a function of the type `... -> Bool` then we know we can never plug in arguments to end up in `Int` so we may fail early. If no goal is known, we just supply a fresh (type) variable.

A `Candidate` is a combination of the AST generated, and the type of that AST. It also includes a so-called `SuggestionBuilderState`. For now, just know that this state is used to backtrack. We will elaborate on the

state in section 7.1.3.

We want the suggestion to use as many of the input tokens as possible. So whenever we see a function, we will keep greedily plugging in arguments until this is no longer possible. Since we want to call `generateExpressionSuggestion` recursively, this poses an issue. Take:

```
trice f x = f (f (f x))

fun = trice plus 4 5
```

when fixing the parenthesis for `fun` we want to call `generateExpressionSuggestion` on the first argument of `trice`. If we generate a suggestion for the remaining tokens `plus 4 5`, we will consume all these tokens and be left with an `Int`. However, `trice` wants a function as its first argument. We want to be able to partially apply `Plus`. so instead of generating one `Candidate` we want a list of candidates. e.g., for `plus 4 5` we would like the following candidates:

```
[ (plus 4 5 , Int               , state)
, (plus 4   , Int -> Int        , state)
, (plus     , Int -> Int -> Int , state)
]
```

Then when plugging in the argument to `trice` we pick the longest one that fits. (In this case `plus 5`.) This is why we get a `[Candidate]` as the result.

The `[Candidate]` input is simply an accumulator to store these intermediate results. The `Maybe Type` input is to keep track of whether we are plugging in arguments to a function. In that case, the type of the function is in the Just. If we are not plugging in arguments into a function, this will be `Nothing`.

In detail. Now that we know what `generateExpressionSuggestion` is supposed to do, we cover in detail how it does so based on the ideas above. We call our arguments: `goal :: Type`, `currentProcessType :: Maybe Type`, and `accumulator :: [Candidate]`. First we check if our `currentProcessType` is a function type or `Nothing`.

If we are not processing a function, we start by getting the type and expression of the next token. This can either be a function/constant name or a primitive (like `42` or `True`). It could also be a lambda or let expression but we cover those in later sections. If we encounter a token without a type (like an equals sign or parentheses), we just throw it out. Regardless of the type we get, we add the expression and type to the accumulator. Unless the type is a function, we are done. If the type is a function, we recursively call

`generateExpressionSuggestion` with the new accumulator and with the function type as `Just` for `currentProcessType` but we keep the same goal.

If we are processing a function, we split up the type of that function into the argument type and return type. We can recursively call `generateExpressionSuggestion` with the argument type as the new goal and an empty accumulator and `Nothing` for `currentProcessType`. This will give us a list of candidates for the next argument to plug into our function. To figure out which candidate to plug in, we pick the longest one where the type fits the goal (there is a most general unifier). We make sure to backtrack if needed. We apply the most general unifier we got from "fitting" our goal (this is handled by the suggestion builder monad; see Section 7.1.3). We construct an application from the last thing added to our accumulator (the function) and the argument found. Then we add this to the accumulator. Finally, we recursively call `generateExpressionSuggestion` with the new accumulator, the old goal, and a new `currentProcessType` where we plug in the return type of the function.

7.1.2 Suggestion for Functions

We want to generate suggestions for entire functions, not just expressions. We call this `generateFunctionSuggestion`. The main challenge is that inside the function, the type environment can be different from outside the function. The arguments are added to the type environment inside the function. The arguments are even allowed to shadow existing functions/-constants. These arguments will be assigned to fresh type variables unless a goal is given. If we already know what the type should be, we can plug in the arguments from the type. For example, take:

```
1 fun :: Int -> Bool -> Int
2 fun x y = ...
```

We know that `x :: Int` and `y :: Bool`. So we plug those in instead.

While inserting into the type environment, we make sure that any variables that shadow (use the same name as) existing bindings, will be placed so they take priority. This is mostly handled by how we structure our type environment; we will elaborate on this in section 7.1.3 once we have consumed the tokens for the function names, arguments and the `=`, we call `generateExpressionSuggestion` with the goal, either a fresh variable or from the type annotation (without the arguments), the other arguments remain empty. Then we pick the longest candidate it gives us that fits the goal. Finally, we combine it, the function name, and the argument into the AST representing a function.

7.1.3 Suggestion builder monad

We use a so-called `SuggestionBuilder` monad. It is responsible for a few things. It holds the token stream we are generating a suggestion for, it holds the type environment, and it holds a counter to generate fresh variables. We have equipped it with a variety of getters and setters to access specific information, but it also lets us define more involved functions, like `applySubstitutionToSuggestionBuilder` which does exactly what it says on the tin. This allows us to abstract away some of the more tedious operations that occur frequently (functions like `consumeWhitespace`, `getToken`, `insertIntoTypeEnvironment` and `getTypesFromState`).

To evaluate our, `SuggestionBuilder` we can run it by giving it the current fresh variable counter and the list of tokens.

`SuggestionBuilder` is implemented as a state monad that will result in an `MaybeError`. A `MaybeError` is just a `Maybe` where the `Nothing` case includes a string as an error message.

SuggestionBuilderState The state looks as follows:

```
data SuggestionBuilderState = State
  { getTokensFromState          :: [TokenInfo],
    getEnvironmentFromState     :: SuggestionBuilderTypeEnvironment,
    getFreshVarCounterFromState :: Int,
    getBranchCounterFromState   :: Int, -- for performance
    getScopeFromState           :: Scope -- depth followed by current branch
  }
```

The `[TokenInfo]` is the list of tokens we are currently processing. The `getFreshVarCounterFromState` holds a counter that we use and increment each time we make a fresh variable; this ensures their uniqueness.

Type environment The type environment is defined as follows:

```
type SuggestionBuilderTypeEnvironment =
        Map.Map String (Map.Map Scope TypePromise)
```

It is like a mapping from function/constant names to types. But instead of one type, we have a mapping from scopes to types. A scope is a combination of two integers. One for the branch; we only need that later and explain it in section 7.4.1. And one for the depth. The depth indicates how deep in the indentation a variable is defined. In this manner, arguments of a (top-level) function are one deeper than globally available functions. This allows us to shadow variables, as we always take the one with the deepest

depth. We add them instead of overwriting the originals so we can easily backtrack once the suggestion builder leaves a specific scope. (Although we will not see this until Section 7.4.) Another reason we do not overwrite is so that we easily apply substitutions to shadowed functions/constants.

For now, we assume a `TypePromise` is just a type (it usually is). In section 7.4.1 we cover when this is not the case.

7.2 Fixing argument order

We have now seen how to fix parentheses. Now to introduce a new problem to solve: argument order. As an example, we take:

```
1 invertNum :: Bool -> Int -> Int
2
3 fun = invertNum 6 True
```

When stepping through this with the algorithm above, we quickly get stuck at trying to plug in `6` in the function that expects a `Bool` first. This does, of course, not work as `6` is an `Int` and not a `Bool`. The problem here is that the function `invertNum` gets the right kind of arguments but in the wrong order. Our assumption is that if we can swap (some of) the arguments to a function and have it type check, then the user misremembered the order of arguments to said function.

To implement this, we do the following. Whenever we find an argument that does not fit in the function, we see if it would fit later. Here we are assuming we will give more arguments (we might not, as we allow for partial application). If we find the argument would fit later, we modify the type of our function. In our example `Bool -> Int -> Int`, we move the type that we did find, in our case `Int`, to the front. We start treating this `invertNum` like it has type `Int -> Bool -> Int`. Doing this, we can immediately plug in what we found and continue generating our expression as normal. Once that is finished, we must take care that we undo this moving to the front. Here we need to fail if the function did not get enough arguments to make the unmoving possible.

In detail. For our code, this is where the `currentProcessType` argument to `generateExpressionSuggestion` is used. We can use it to store these temporary swappings in the type of the function we are processing.

So whenever we fail to plug in an argument, the first thing we do is recursively call `generateExpressionSuggestion` again, with a free variable as the goal since we just want to know what the type of the next argument will be. Then we go over the candidates one by one, taking the longest first and seeing if it will fit in any of the other arguments. When doing this, we

make sure we look at the arguments from left to right to preserve relative function order, e.g., if you have `fun 4 True 5` the `4` will always stay to the left of `5`. If we find where it fits, we move that argument to the front of the function, keeping track of the index. Then we can continue with this new type as if normally plugging in arguments. However, the expressions this generates still contain applications with wrong orderings. So we go over all the candidates generated and swap the application based on the index we stored earlier. Since some of these will be partial applications, this might fail. In that case, we just remove them from the accumulator. Taking special care to not remove candidates from before the arguments did not fit, as these partial applications are perfectly valid.

7.2.1 Infix Operators

Support for infix operators is out of scope for this thesis. However, this is a good time to explain why. The main problem with infix operators is that our current approach reads from left to right, so we will build a suggestion for an argument for which we do not know the (type of the) function it will be applied to yet. We foresee a few ways of potentially resolving this issue.

The first is by using some preprocessor that makes infix functions prefix. This can come with some challenges, especially if the parentheses on the left of said infix function are wrong. But it might support some cases that otherwise would not work.

The second approach, similar to the previous, is using a normal parser whenever we see an opening parenthesis, instead of ignoring it like we do now. This means that when we encounter parentheses, we first assume they are correctly placed and use the normal parser and inferencing algorithms to figure out their type. Only using the suggestion builder when this fails. This would mean that if the mistake is not in the section with infix operators, we can still give a suggestion, as the infix operators will never be touched by our suggestion builder. In fact, this would go for any unsupported language construct.

The final (third) approach would be that whenever we find an argument we cannot fit, we also try and see if we can find an infix function that takes that argument after it. This approach leaves the most room for clever algorithms.

7.3 Lambda Expressions

Now to add lambda expressions. Considering we already covered how to generate suggestions for functions, there is not much special about lambda expressions. We still need to keep track of the scoping, etc., but that can be done in a similar manner. We also need to make sure we remove the arguments from the type environment when we are done.

The most notable is that we support a few cases where the user makes mistakes regarding the `->` symbol. For example, if the user writes:

```
trice f x = f (f (f x))

plus :: Int -> Int -> Int

fun = trice \x = plus 5 x 8)
```

We have a problem since the `=` should have been an `->`. We support this by taking arguments until we see a symbol that is not a name. Here `=` is such a symbol. Then we only try to consume an actual `->`, if it is not there; we just act like it was. So in our example, the `=` will still be there when we try to build a suggestion for the expression. Since the `generateExpressionSuggestion` ignores all unknown tokens, this is only then ignored.

We can do even better. If we know the goal must be a function with a set number of arguments, we only try to take up to (we may have fewer) that number of arguments. This allows us to insert the `->` in situations like this:

```
iterate :: (Int -> Int) -> Int -> (Int -> Int)

plus :: Int -> Int -> Int

fun = iterate \x plus 5 x 8 2
```

In detail. Whenever the `generateExpressionSuggestion` tries to get the type of the next argument, we might encounter a `\`. Instead of ignoring it, we go into a special `processLambda` section of the suggestion builder. This section is responsible for consuming all the tokens of the lambda and returning its type so it can be used like any other function. We check the goal and see if we know how many arguments are needed (this gives the argument limit). Note here that if the goal type ends on a variable, we cannot say how many arguments the lambda should take. Then we consume name tokens until we either find a non-name symbol (such as `->`) or when we hit the argument limit. We make sure to add the types of the arguments to the environment, making fresh variables if the arguments were not in the goal. Then we try to consume a `->` if it is not there, we just assume that it was forgotten and continue. Then we can call `generateExpressionSuggestion` recursively with the goal either got from the goal or a fresh variable. Then we pick the longest candidate that fits the type. We can now build up the expression and type of the lambda expression. Finally, we make sure to

remove the arguments of the lambda from the type environment.

7.4 Let Statements

Adding let expressions comes with two significant challenges: processing order and indentation.

7.4.1 Processing Order

Until now, we have been processing the tokens from left to right. Let expressions pose a problem. Take:

```
plus :: Int -> Int -> Int

fun x =
    let
        fun3 = fun2 x 5
        fun2 x1 x2 = plus (plus x1 x2) 7
    in
        fun3)
```

The only mistake here is the `)` at the very end. If we process the local definitions in order, we first try to make a suggestion for `fun3` but when we do this, we get stuck. In the expression of `fun3` we use `fun2` but we have not seen `fun2` yet. So we have no idea what its type is. This problem occurs since `fun3` is allowed to depend on `fun2` which is defined later.

To solve this, we need to know that `fun2` exists before we process `fun3`. Whenever we see a `let` keyword we have to look ahead to find all the local definitions. We do this by splitting the remaining tokens on the pattern:

```
Newline, Whitespace, some number of Names, EqualsSign
```

where the first name is the function name and the rest are its arguments.

For now, we assume there is just one let. For multiple lets, it gets more complicated as indentation starts mattering. We cover this in section 7.4.2.

Once we find all the local definitions, we need to decide on an order to process them. Normally this can be solved by dependency analysis. However, this requires us to know the structure of each definition, which is the problem we are trying to solve in the first place. A simple approach where we just check to see if one of the tokens uses a different function by its name also does not work, as we are allowed to shadow variables. Whenever we see a name token, we cannot be sure to what it is referring without knowing the structure of the code.

Instead of dependency analysis, we just start at the first one. Then whenever we find a function that is defined later, we jump to build a suggestion for that one. Once done with that, we jump back.

Jumping and branching. Jumping to generate a suggestion for something else mid-suggestion and jumping back comes with some challenges.

The first is when to jump. We solve this by storing a `TypePromise` instead of a normal `Type` in the type environment. A `TypePromise` is either a `Constant`, `Function` or a `Result`. A `Constant` is just a type, a `Function` holds all the information required to start building a suggestion at a certain point, and the `Result` holds the type after building a suggestion for a `Function`. It also holds the other results of generating the suggestion to avoid double work later.

So whenever we process a let expression, we first populate the type environment with these `Function` type promises. Then whenever we request the type of a `Name` token from the type environment, if we get a `Function` type promise, we jump to evaluate that.

Now we know when to jump; we need to know how to jump. The challenge here is in the type environment. At the time of jumping, we might be in a deeper scope with more types available. We need a way of removing them from the environment when we jump. You might consider copying the type environment known at the time of adding the `Function` type promise. However, we need to keep using the most recent type environment since we might have learned some types are more specific before we jump. Another problem is that any restrictions found while doing the jump need to be applied to the type environment once we jump back.

Take the following example:

```
1  plus :: Int -> Int -> Int
2
3  fun x =
4      let
5          fun3 =
6              let
7                  y = x
8              in
9                  fun2 y 5
10         fun2 = plus x 7
11     in
12         fun3)
```

When we jump to evaluate `fun2`, we must be careful to remove `y` from the type environment, as it should not be available in `fun2`. However, while

building a suggestion for `fun2` we learn that `x` should have the type `Int`. We can plug that in by applying the relevant substitution to our type environment, but once we jump back, we have to reintroduce `y` to our type environment. But since we now know that `x :: Int` and `y = x` we need to make sure this is updated for `y` too, instead of `y` having the type before jumping.

We could solve this by gathering substitutions and splitting the type environment each time we branch. Instead, we introduce the idea of a branch. In the `SuggestionBuilder` monad, we keep track of our current scope. The scope is both the depth and the branch we are at. We explained the depth in section 7.1.3. The branch indicates which universe we are in. Every time we jump, we create a new branch, populating it by taking all the types that are available at the depth of the `Function` type promise (the depth is stored in the `Function` type promise upon its creation) and copying them onto a new branch. Branches are indicated by a number that simply counts up or down upon creation or deletion of a branch. Then whenever we find a substitution, such as learning that `x :: Int` like in our example, we can apply it to all branches. While looking up something from the type environment, we only consider our current branch. When done jumping, we simply unbranch, removing all types at that current branch from the environment.

So by making our type environment a bit more complex, we can simplify managing substitutions and jumping greatly.

Generating the suggestion To summarise, we can add type promises to our type environment containing functions that will be evaluated on demand.

When generating the suggestion, we just add these functions to the type environment and request their types from the top down. This will in turn give `Result` type promises containing both the expressions and the types of these local definitions.

We would like to note that recursive functions are out of scope for this thesis. If one wanted to support them, we recommend taking the original type promise out of the type environment as soon as you jump to evaluate it. This can be done to avoid loops. Alternatively, one could replace it with some different type promise to detect the recursive nature.

Once gathering the results is done, we look at the tokens remaining after generating the suggestion for the last local definition. This should contain the `in ...` part of our let expression. We added a special case for the `generateExpressionSuggestion` such that it will always fail on seeing an `In`. This ensures we never process tokens past it. Once we get the remaining tokens of the last local definition, we try to consume an `In` if it is there; otherwise, we just assume it was forgotten and continue. Finally we call `generateExpressionSuggestion` on the remaining tokens to get the expression of the let. This gives us everything needed to build our let

expression AST and thus our suggestion.

Local definition order matters. There is one problem we do not solve in our current approach. Recall that we are aiming to find the fewest number of changes to make the program type check. Now look at the following:

```
1 invertNum :: Bool -> Int -> Int
2
3 fun x y =
4         v1 = invertNum x y
5         v2 = invertNum y x
6         v3 = invertNum y x
7     in
8         ...
```

Here one of the sets of arguments is flipped. Namely, the first one, `v1`. Swapping the arguments for `v1` gives us the smallest number of changes. However, our current algorithm suggests something different. It first looks at `v1` and then realises that `x` should be a `Bool` and `y` a `Int`. Then for `v2` and `v3`, since it has already decided what y and x should be, it will suggest flipping the arguments for both `v2` and `v3`.

It gets even worse if we imagine `v3` to look like `v3 = plus x x` since this will now fail since in `v1` we already decided that `x` should be a `Bool`.

So the order of the expressions can affect if and what suggestion our suggestion builder will give.

This was one of our major performance considerations. To guarantee we get the minimum number of changes, we would have to try every order of processing the local definitions. Doing this would make the search space very large for big let expressions. We are accepting the risk that we might not generate a suggestion or a sub-ideal one. We deem it worth it to keep the search space small.

We do think there is room for clever algorithms gathering restrictions to converge to the optimal solution, but we leave this out of scope.

7.4.2 Indentation

Let expressions rely on indentation. It is important that all the local definitions of the same let start at the same indentation. This is especially important when let expressions are nested. Our problem is as follows: people tend to make mistakes in indentation, and we would like to make suggestions that can fix these mistakes.

As seen in the previous section, we use a pattern to get all the local definitions. For processing nested lets, we need to figure out a way to determine

which local definitions belong to which let. We could have gone for an approach where we look for the `in` token to define the end of a let expression. We chose not to do this for two reasons. The first is that there are many cases where we do not need the `in` and can even make a suggestion telling where it should go if it was forgotten. The second reason is that we wanted to find a solution that could also work for case statements, which do not have a token to denote their end. Unfortunately, case statements fell out of scope, so we cannot comment on if our algorithm can actually help with case statements.

Instead of relying on the `in`, we try to group local definitions with their let. When splitting on the

```
1 Newline, Whitespace, some number of Names, EqualsSign
```

pattern, we also check if any of the tokens are a `let` token. We end up with a list of tokens that might or might not contain a let. We want to determine which of these local definitions belong to the let we are currently processing. Since we cannot rely on indentation, we are forced to go over all the possible configurations.

Configurations must be valid. For example, if one local definition contains a let, the next local definition must be part of said let expression. Another restriction is that we cannot have a local definition on its own not belonging to the let we are processing, unless the local definition before has a let. We only want to know which local definitions belong to the let we are currently processing. To generate all possible configurations, we must go over every nested let and determine how many local definitions after it belong to it. The minimum is one. At a maximum, it can take all the local definitions after it. When we have multiple lets in the local definitions, this "take all the local definitions after it" allows for duplicate configurations. e.g., if we have a list of local definitions with two nested lets, we can have one situation where the first let takes every local def until the end. And one situation where the first let takes everything up to the second let, and the second let will take the rest. Since we only care for which definitions belong to the let we are currently processing (let us call it let number 0 in our example), we see these two situations as the same. So if there are multiple nested lets, we can set the maximum number of local definitions each let may take to stop at the next nested let. In short, we calculate the possible configurations by letting nested let take at least one and at maximum till the next nested or end of file. We go over all these possible ranges.

We cannot rely on the indentation, but we can use it to guide our search. So for every configuration, we check the character positions of the function name tokens. Here it is especially nice we have actual position information from the source files, as indents and detents can hide the size of the change.

It is better to remove one space to remove an indent than to add 4 to add one.

For each of the configurations, we calculate how many character positions should be changed to make the indents work. Then we go through them one by one, with the smallest difference first. The main advantage of this approach is that if the indents are already correct, we immediately try the correct configuration. We keep trying configurations like this until we find a configuration that works. The biggest downside is that if we cannot generate a suggestion due to some other problem, we will still try all combinations. We believe there is a way to detect if a certain problem is not fixable by different indentation and then stop or search more effectively, but this is out of scope.

7.4.3 Summary

All this together, we get the following algorithm for handling lets.

First, we split the remaining tokens using the

```
Newline, Whitespace, some number of Names, EqualsSign
```

pattern. Then we use the position information of the name of each section to generate all the possible configurations of local definitions belonging to let expressions. We go through them one by one, trying the one with the smallest indentation change first. We add all the local definitions as `Function` type promises to our type environment. These `Function` type promises tell it how to call `generateFunctionSuggestion` with the right tokens. Then we request all their types in order from top to bottom. This causes the `Function`s to be evaluated and return with types and expressions. We continue with the tokens left over after the last local definition. Then we try to consume an `in` token. And finally, call `generateExpressionSuggestion` to get the body of the let.

Dealing with the type promise. Whenever we request a type from the type environment, if it is a `Function` type promise, we branch and evaluate the function. We replace the `Function` type promise with a `Result` type promise, which stores the output so we will not have to compute it again. Then we make sure to unbranch so we can continue where we were before branching, now with the type of the function that was not evaluated before we branched.

Chapter 8

Displaying the Differences

When our suggestion builder is done, it gives us an AST. However, we need to display this information to the user. We could simply convert the generated AST to text and give this to the user. This poses a few problems:

The first is the lack of comments in the generated AST. The suggestion builder ignores comments in generating its suggestion. This is especially undesired whenever the suggestions would remove comments from a part of the suggestion that was correct (e.g., a local def in a let expression when the problem is after the `in`).

The second is that there are multiple ways to write the same AST. If the problem is in one place, we do not want to change the syntax where there was no problem to alternative syntax that generates the same AST.

The third is that it is hard to find what has changed compared to what the user typed. Since our suggestions are only that, suggestions. The suggestions could be wrong, all generated suggestions type check, but this does not rule out logic errors. So the user must check if the suggestion is correct. To do this, we want to emphasise the differences between what the user typed and what the suggestion builder suggests. We display this by giving colours. green for tokens that should be added, red for removed. (Probably different colours for red-green colour-blind people.) We also want to display a version with only the new code to make copy-pasting easy.

8.1 Generating the diff

Our algorithm works in two steps. In the first step, we convert the AST into a list of so-called target tokens. A target token is a token that is either required or optional. As an example, take the AST `Application (Name "not") (True)`, this can be written as:

```
not True
```

or as

```
not
    True
```

(We omit the case where we use superfluous parentheses here to keep the example small.)
So the expected tokens would be:

```
[Required (Name "not"), Optional newLine (Just 1), Optional Indent (Just 1),
   Required True]
```

We are saying the newline and the indent are optional. The `Just 1` here is used to signal that if you use the `Newline` you must also use the `Indent`, as it has the same identifier. Having these optional tokens solves the second issue of being able to write the same AST in different ways.

For the second step, we find the fewest number of edits in the original token stream to end up with something that matches these optional tokens. We convert each token of the original string into a so-called *action token*. An action token is either Keep, Remove, or Add for some token. We calculate this using a variation on the Levenshtein distance algorithm [Yujian and Bo, 2007]. For each token, we choose either to add it, remove it, or keep it (if the action token and user token are the same). For optional tokens, we do not add weight for not using the token, and if we do use it, we make sure to also use all other tokens with the same number. We always keep comment tokens, even if not in the optional tokens; this solves the first problem with comments.

We can then print all the Add tokens in green, keep tokens in white, and remove tokens in red to get the desired difference. We can print only the Keep and Add tokens in the default text colour to get the final code that is suggested.

8.1.1 Performance

The modified Levenshtein distance diffing algorithm goes through a huge search space. It is therefore quite slow. Using the GHCi profiler, we found that (for some basic test input with small functions) the token difference calculation takes 92.4% of the runtime and 98.9% of the memory. (See Appendix B for the full input.) We used a dynamic programming approach to prevent the algorithm from doing double work. Unfortunately, it is still

much slower than the suggestion builder. To resolve this, we would need a better diffing algorithm. However, this is out of scope for this thesis. The focus of this thesis is on the suggestion builder (Chapter 7), not so much on the implementation details of this diffing algorithm. However, a modified version of Myers' Diff Algorithm [Myers, 1986] seems like a promising approach.

8.2 Target-Tokens generation

In this section, we give how we are converting the AST into target tokens (see Section 8.1 for what target tokens are). We will go over all the expression constructs in our language (see Chapter 5) and show how you make the tokens. In this way, we define a `createTargetTokensFromExpr` function.

Constants and functions For constants such as Integers/Booleans or Function name, we can simply require the corresponding token.

Parenthesis For parenthesis that contains some `AST`

```
[Required Lpar] ++ createTargetTokensFromExpr AST ++ [Required Rpar]
```

Function Application for applications where we apply `AST2` to `AST1` which normally looks like,

```
AST1 AST2
```

where the space indicates function application. One can also write

```
(AST1
    AST2)
```

(with or without the parenthesis)
We can use

```
[Optional Lpar (Just bracketId)]
++ createTargetTokensFromExpr  AST1
++ [Optional Indent (Just indentId), Optional Newline (Just indentId)]
++ createTargetTokensFromExpr AST2
++ [Optional Rpar (Just bracketId), Optional Dedent (Just indentId)]
```

Here `bracketId` and `indentId` are both numbers generated uniquely. (We use a counter which we increment each time we need a unique value.)

This allows for extra parentheses around the application and for the argument to be given on the next line as long as there is an indent (and dedent).

Lambda Expressions Lambda expressions are a bit more complicated. In the AST, lambdas always have one argument `arg1` and one expression `AST`.

```
(\arg1 -> AST)
```

However, we can also have nested lambdas that are written as

```
(\arg1 arg2 -> AST)
```

or as

```
(\arg1 -> (\arg2 -> AST))
```

We assume the choice between the two is a conscious decision by the user, in the same way one might write unnecessary parentheses to convey meaning. So we want to support both.

To do this, we need to look at how many arguments we have. We recursively look into the AST, and if there is a lambda in a lambda, we process them into Target-Tokens at the same time. This gives us a list of arguments. Taking the example above, we see that `-> (\` and an ending `)`, are optional after each argument except for the last argument; there it is mandatory.

Doing this, we end up with:

```
[Require Lpar, Require Lambda ]
++ Require (Name argumentName1)
++ [Optional RArrow (Just lambdaID1), Optional Lpar (Just lambdaID1), Optional
    Lambda (Just lambdaID1)]

++ Require (Name argumentName2)
++ [Optional RArrow (Just lambdaID1), Optional Lpar (Just lambdaID2), Optional
    Lambda (Just lambdaID2)]

...

++ Require (Name argumentNameN)
++ [Require RArrow ]
++ [Optional Indent (Just newLineId), Optional Newline (Just newLineId)]
++ createTargetTokensFromExpr AST
++ [Optional Rpar (Just lambdaID1)]
++ [Optional Rpar (Just lambdaID2)]
..
++ [Optional Rpar (Just lambdaIDn-1)]
++ [Require Rpar]
++ [Optional Dedent (Just newLineId)]
```

Here `lambdaID1 ... lambdaIDN` and `newLineId` are uniquely generated. The `newLineId` allows for the newline after the `->` like so:

```
(\arg1 arg2 ->
    AST)
```

We currently do not allow for next lines after the optional arrows in between; this should probably be added, but due to time constraints we have not.

Let Expression Let expressions pose a problem. Have a look at the following example:

```
plus 5 (let x = 4
            y = 5 in plus x y)
```

this is correct. The problem is as follows:

```
plus 5 (let x = 4
    y = 5 in plus x y)
```

is not correct since the `x` and the `y` are not aligned. However, we are just using `Indent` and `Dedent` tokens, so we cannot distinguish between the two as they both have just one indent. This reveals a fundamental issue in how we chose to represent our tokens. This requires a different way to handle indentation. However, this is out of scope for this thesis; instead we use a workaround. We always force the 'let' and the first local definition in the 'let' to be on a new line, like so:

```
plus 5 (
    let
        x = 4
        y = 5
    in plus x y)
```

This ensures we cannot run into the issue above, and the suggestions always type check.

In our AST, a let expression consists of a list of local definitions and a body `AST`.

```
[Require Indent, Require Newline, Require Let, Require Indent, Require Newline]
++ concat definitionTokens
++ [Require Dedent, Require In, Optional Indent (Just firstIndentId), Optional
    Newline (Just firstIndentId)]
++ createTargetTokensFromExpr AST
++ [Optional Dedent (Just firstIndentId), Require Dedent]
```

Here `definitionTokens` is a list of tokens built for each local definiton. Each local def has a name `name`, a list of arguments `arg1,arg2,..., argn` and a function body `LocalAST`. For each local definition, we convert it into tokens like so:

```
[Require (Name name)]
++ [Require (Name arg1)]
++ [Require (Name arg2)]
...
++ [Require (Name argn)]
++ [Require EqualsSign]
++ createTargetTokensFromExpr LocalAST
++ [Require Newline , Optional Newline Nothing]
```

The final `Optional Newline Nothing` allows for white lines after each definition.

Chapter 9

Performance

In this section, we cover the performance of the system. Since we actually have an implementation of the algorithm, we start with some actual runtime analysis. After that we look at the complexity of our algorithm and how we expect it to scale.

We would like to point out that in the best case for the system, there are no errors. If there are no errors to fix, the performance impact of the suggestion engine is essentially 0 since it will not be called. So performance should not be a concern when considering suggestions for compilers that need to compile large codebases of known working code.

9.1 Runtime Analysis

Hardware Specifications: We ran the tests on a 13th-gen Intel i9-13900K with a base clock of 3 GHz and a boost up to 5 GHz and 32 GB of dual-channel DDR4 memory running at 3200 MT/s.

We use the GHC profiler [ghc,] to measure runtime, all the timings are according to the GHC profiler.

When measuring the performance, we ran into a very clear bottleneck. As elaborated on in section 8.1.1 the token difference algorithm is very slow. For example, generating the suggestion for the code in Appendix B takes 0.89 seconds. This includes parsing and type inferencing. If we bypass the difference algorithm, it is reported as taking 0.00 seconds. While this version does give (nearly) the same suggestion as the version with token differences. The nearly refers to this version not respecting unnecessary parentheses, as the bypass simply prints the required tokens and does not do the diffing with the user tokens.

Since the token difference algorithm is so slow, and since our focus is on the suggestion builder, we will be bypassing the differencing algorithm in further runtime analysis.

As a stress test, we put all the examples from Chapter 2 in a single file to

run them all at the same time. This takes 1.52 seconds with the differences and 0.01 seconds with it bypassed. With the differences bypassed, we can measure the relation between the suggestion builder and the normal compiler phases. We see that we spend about 20% of the runtime on generating the suggestions. and about 8% of the memory (of the roughly 14 megabytes total). We expect this to be a much smaller percentage in practice, as for most of these examples we do not get to the type inferencing phase, and we would expect only a few functions to have problems, as opposed to all of them in this case.

As we see in section 9.2 that there are situations where theoretically the complexity can get out of hand. However, we struggled to construct an example where this leads to significantly increased runtime. We therefore also deem it unlikely for this to be found in practice.

Overall, we are quite happy with the performance; the performance with the differences is acceptable, and the performance without the differences is excellent.

9.2 Algorithm Complexity

We have seen from testing that the runtime is acceptable. In this section, we zoom into the suggestion builder and project how it would scale to much larger programmes.

Here, we would like to make a distinction between the case where the user has only made one mistake. And the worst case, where multiple errors can take the most out of our errors.

9.2.1 One Mistake

If there is only one mistake, our algorithm behaves mostly linearly. For all language constructs, if there are different things to try, we first look at the case where we assume the source code was correct. The only place where this is not linear is in processing nested let expressions. For nested let expressions, we currently build all possible configurations of local definitions belonging to `lets`. Then we sort it based on the difference in indentation required. If the mistake was not in the indentation, the first configuration we try will be the correct one. However, the work needed for generating and sorting the possible configurations depends on the number of nested let expressions and local definitions. In practice, one needs a really large number of local definitions for this to become an issue.

Once the issue is ”found”, the worst case is for swapping arguments where we have to re-compute the argument with a free variable as a goal. This doubles the work at worst.

9.2.2 Worst Case

When there are multiple issues, we can compound the worst cases from the previous section. If one uses a lot of nested lets with the worst indentation and consistently the wrong order of arguments, one can probably approach exponential runtime. However, one of our assumptions is that a user writes mostly correct code. So we deem this unlikely to cause issues in practice. One would have to do quite a few things wrong to make the runtime explode.

The worst cases worth worrying about are all the cases where we currently fail to build a suggestion in the first place.

Chapter 10

Conclusion

We have provided a small functional core language with types, function definitions/applications, constants, lambda abstractions and let expressions. Which serves to demonstrate our algorithm. We have given the suggestion builder algorithm such that it can generate type-checking ASTs based on the type system and possibly user-provided type annotations. We have seen how these suggestions can solve mistakes in parentheses, argument order, lambda abstractions, indentation, and the definition of let expressions. We have seen how these mistakes were iteratively supported by the suggestion builder and how we support these mistakes being combined. We have shown how these suggestions can be displayed to the user by generating the differences between the given tokens and the AST generated as our suggestion. Here we made a tradeoff between always finding the smallest number of changes to a compiling problem and runtime. We ended up with an algorithm that would be performant enough for integration in compilers. We stuck to the requirements of always generating type-checking suggestions and leaving working code unchanged.

We also provided instructions on how these suggestions can be integrated into compilers. even if such a compiler supports a bigger language than the suggestion builder.

This concludes our contribution towards the main problem of finding the minimum number of transformations to make some malformed code compile. Here our transformations include adding indentation, keywords (such as `->` and `in`) and parentheses and the changing of argument order. And although we have forgone guaranteeing the minimum number of changes in favour of performance, we still get reasonably close in most examples.

Chapter 11

Reflection and Future Work

11.1 Reflection

We started this research with a much larger scope in mind. We also envisioned supporting ADTs, case statements and list notation. This turned out to be way too much. The first major time sink was building a compiler to hook this suggestion builder into. We are happy we made this decision; the alternative was to adapt an existing compiler. We fear that familiarising ourselves with an existing compiler would also have been a significant time investment. In addition, we are happy our compiler is quite minimal. We fear that having to support more primitives and language constructs would have introduced more work without significantly contributing to how we generate our suggestions. We also value being able to change the compiler with relative ease. This has allowed us to break the compiler up into sections that might fail. We fear this would have been significantly harder with an existing compiler.

Another thing we underestimated was the management of the type environment and substitutions. Making mistakes in this area can lead to really subtle bugs.

We did find that once a good structure was found, adding more suggestions became relatively easy. For example, once we figured out functions and expressions, adding lambda expressions only took us roughly a day.

Let expressions really turned out to be much more difficult than anticipated. Initially, this was meant as a one-week project to prepare for case statements. But handling mistakes in indentation and the dependencies took much longer to figure out. We are happy we decided to tackle these issues. At some point we considered not solving the indentation issue and only generating suggestions for mistakes in the local definitions or the body of the let. This would have been significantly easier and would have probably allowed for time to add case statements. However, we feel the suggestions would be worse off for it.

Overall, we would have hoped for a more feature-complete language but do feel like we have a strong foundation for generating high-quality code suggestions.

Acknowledgements We thank the Radboud University for providing facilities and maintaining an AC system. We thank the Software Science department of the Radboud University for attending presentations and providing feedback. Most of all, we thank dr. P.M. Achten for the biweekly meetings and invaluable feedback and help. And thank our second assessor, dr. M Lubbers for offering to review this thesis even during his holiday.

11.2 Future work

For future work, we have roughly three categories: refinements on our current approach, additions to our current approach, and other directions to take the problem in.

All of this aims to solve the question, "What is the smallest number of transformations to some malformed program such that it compiles?". Most importantly, we would like to see more research towards this question.

11.2.1 Refinements

(Mutual) Recursion We currently do not support suggestions for (mutually) recursive functions. We feel like there would be a lot of easy ground to gain here.

Let Statements Our current approach to building suggestions for let statements is a little rough around the edges. For indentation, we currently try all combinations. This can probably be refined by looking at why generating a suggestion failed and using that information to change indentation in a more guided manner. e.g. if a suggestion failed because a function called `fun3` was not in scope, but we do have that as one of the local definitions. Then it might be wise to change the indent of that specific function. We can also imagine detecting faults that cannot be solved by changing the indentation. When that happens, we should forgo trying all other indentations.

Another small addition to let statements is to check the remaining tokens of not only the last section, but of all the sections before the last one as well. If there are proper functions to be generated from them, the user probably missed a next line and has a local definition that we missed. We can miss such a definition if it is define don the same line as another. The pattern we use to split definitions requires them to be on a new line. Getting two local definitions on the same line is a mistake that could happen during refactoring, and it would be nice if our suggestions could deal with that.

Token Differences Our current token diffing algorithm is really slow. This should be reworked; a variation on [Myers, 1986] looks promising. One could also forego the step to Target-Tokens and generate the diff based on the input tokens and the generated AST directly.

Representation of Swapping Arguments Currently, when our suggestion involves swapping arguments, we see that in the diff as one argument being removed and added again. This is usually detectable as swapping arguments. But this conclusion requires effort from the user's side. It also fails to be clear on what should be moved, should `a` be moved after `b` or should we move `b` before `a`. It would be nice if there were some other notation to clarify to the user that certain arguments should be swapped.

11.2.2 Additions

Infix Operators Adding support for infix operators can be a fun challenge. We suggest either a preprocessor to make the infix functions prefix or a modification to the failure case of suggestion generation to support said infix operators.

Case Statements and ADTs The next logical language features to add are case statements and algebraic data types. Our hope is that this can be achieved with minimal changes to the current structure of the suggestion builder. But this remains to be seen.

List Notation Lists have unique notation inherited from mathematics. Lists are also commonly used. We would like to see some extra effort put into making good suggestions for list notation.

Type-driven spelling suggestions Sometimes we cannot find some name in the type environment. This usually indicates a spelling error. Spelling error suggestions are usually given based on Levenshtein distance. We can do better by only looking at functions which types would fit.

Taking this concept even further, whenever we find arguments that do not fit a function, we can look for functions with a similar name where arguments do fit. We can have special dictionaries for defining what "similar" is in this case. For example, having different maps be closely related so we can say that e.g. `map` and `mapTree` are closely related. We can even have mappings where we tell that e.g. `get` is related to `lookup` as in other languages `lookup` is often called `get` .

Compiler Integration It would be nice to see these suggestions be integrated into an actual compiler. In doing so, one might want to use the

existing parser/inferencer in the suggestion builder to basically try if the normal parser/inferencer can figure out some section of the code and only using the suggestion builder if it cannot. This can help support more language features as long as the problem is not in an unknown feature.

Holes and Program synthesis In program synthesis, it is common to work with a so-called hole. Where, with a hole, you basically signal the compiler to just fill something in. One can imagine extending the suggestion builder with this capability, especially if one considers typed holes. A typed hole signals "give me some expression" with a certain type.

We could even plug in holes with unused variables, if the types match.

Another use of holes would be not to plug them in, but to expose them to the user. So if a function really needs an `Int` as input which we do not have, instead of failing, we can just add a hole there and let the user input their desired `Int`.

11.2.3 Other Directions

Suggestion builder combinators For parsers we have been using parser combinators for a while. In a way, the suggestion builder is similar to a parser. One can imagine constructing suggestion builders in a similar way. Where there are some primitive suggestion builders for the pprimitives and ways to combine them and give them restrictions. The goal would be to be responsible for both parsing, type inferencing and building suggestions at the same time. Where we try to hide how these suggestions are generated as much as possible.

Editor Integration One can imagine a system where suggestions are directly integrated with a code editor, making it easy to accept and view suggestions.

AI Large language models can also be used to generate suggestions to some extent. The main problem is that they are often unreliable[Popovici, 2023]. One can imagine a hybrid approach where an LLM is used as a pre-processor before the suggestion builder. Or as a system where the suggestion builder can call on the AI as soon as it gets stuck.

Bibliography

[ghc,] 8. profiling — glasgow haskell compiler 9.12.2 user's guide. [Online; accessed 2025-07-11].

[ale,] alex: Alex is a tool for generating lexical analysers in haskell. [Online; accessed 2025-07-11].

[Becker et al., 2019] Becker, B. A., Denny, P., Pettit, R., Bouchard, D., Bouvier, D. J., Harrington, B., Kamil, A., Karkare, A., McDonald, C., Osera, P.-M., et al. (2019). Compiler error messages considered unhelpful: The landscape of text-based programming error message research. *Proceedings of the working group reports on innovation and technology in computer science education*, pages 177–210.

[Bruch et al., 2009] Bruch, M., Monperrus, M., and Mezini, M. (2009). Learning from examples to improve code completion systems. In *Proceedings of the 7th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on the foundations of software engineering*, pages 213–222.

[Campos et al., 2021] Campos, D., Restivo, A., Ferreira, H. S., and Ramos, A. (2021). Automatic program repair as semantic suggestions: An empirical study. In *2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*, pages 217–228. IEEE.

[Franks et al., 2015] Franks, C., Tu, Z., Devanbu, P., and Hellendoorn, V. (2015). Cacheca: A cache language model based code suggestion tool. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, volume 2, pages 705–708. IEEE.

[Hage et al., 2006] Hage, J., Keeken, P., et al. (2006). Mining for helium. Technical report, UU WINFI Informatica en Informatiekunde.

[Heeren, 2005] Heeren, B. J. (2005). *Top quality type error messages*. PhD thesis, Utrecht University. ISBN 90-393-4005-6.

[Hudak et al., 2007] Hudak, P., Hughes, J., Peyton Jones, S., and Wadler, P. (2007). A history of haskell: being lazy with class. In *Proceedings of the*

third ACM SIGPLAN conference on History of programming languages, pages 12–1 – 12–55.

[Jones, 2003] Jones, S. P. (2003). *Haskell 98 language and libraries: the revised report*. Cambridge University Press.

[Lee et al., 2018] Lee, J., Song, D., So, S., and Oh, H. (2018). Automatic diagnosis and correction of logical errors for functional programming assignments. *Proceedings of the ACM on Programming Languages*, 2(OOPSLA):1–30.

[Myers, 1986] Myers, E. W. (1986). An O (ND) difference algorithm and its variations. *Algorithmica*, 1(1):251–266.

[Popovici, 2023] Popovici, M.-D. (2023). Chatgpt in the classroom. exploring its potential and limitations in a functional programming course. *International Journal of Human–Computer Interaction*, pages 1–12.

[Raychev et al., 2014] Raychev, V., Vechev, M., and Yahav, E. (2014). Code completion with statistical language models. In *Proceedings of the 35th ACM SIGPLAN conference on programming language design and implementation*, pages 419–428.

[Rhemrev, 2023] Rhemrev, T. (2023). Creating better error messages by improving their presentation. Bachelor thesis, Radboud university.

[Tirronen et al., 2015] Tirronen, V., Uusi-Mäkelä, S., and Isomöttönen, V. (2015). Understanding beginners' mistakes with haskell. *Journal of Functional Programming*, 25:e11.

[Yujian and Bo, 2007] Yujian, L. and Bo, L. (2007). A normalized levenshtein distance metric. *IEEE transactions on pattern analysis and machine intelligence*, 29(6):1091–1095.

Appendix A

Code

Our full Haskell code can be found on GitHub:
`https://github.com/sijmenvb/masters-thesis`

Appendix B

Runtime Profiling

We used the GHC profiler on the following input:

```
trice f x = f (f (f x))

iterate :: (Int -> Int) -> Int -> (Int -> Int)

invertNum :: Bool -> Int -> Int

plus :: Int -> Int -> Int

fun x y =
    let
        var3 =
            let
                fun3 = plus
            in
                fun4 5 (fun3 5 5)
        fun4 a b = plus a b
    in
        var3)



fun2 = let
        x = 4
    in x
```

Which gave us the following profiling. (We have omitted the source code location to fit it on this page.)

```
    Mon Jun 23 23:49 2025 Time and Allocation Profiling Report  (Final)

       masters-thesis +RTS -p -RTS

    total time  =        0.89 secs   (959 ticks @ 1000 us, 1 processor)
    total alloc = 3,047,060,208 bytes  (excludes profiling overheads)

COST CENTRE        MODULE                        %time %alloc

GenerateActions2  Suggestions.TokenDifference   91.1   98.9
compare           Lexer.Tokens                  3.2    0.0
compare           Suggestions.TokenDifference   2.8    0.0
```

(We also omitted the rest of the profiling file, as it is 2000+ lines long and not interesting.) In comparison, all other major functions in the program get rounded to 0.0.